EnvGen : UGen : AbstractFunction : Object

Envelope generator

Source: EnvGen.sc

See also: Linen, Env

Description

Plays back break point envelopes. The envelopes are instances of the Env class. The envelope and the arguments for `levelScale`, `levelBias`, and `timeScale` are polled when the EnvGen is triggered, and at the start of a new envelope segment. All values remain constant for the duration of each segment.

```
{ PinkNoise.ar(EnvGen.kr(Env.perc, doneAction: Done.freeSelf)) }.play
```

Class Methods

```
EnvGen.ar(envelope, gate: 1.0, levelScale: 1.0, levelBias: 0.0, timeScale: 1.0,
      doneAction: 0)

EnvGen.kr(envelope, gate: 1.0, levelScale: 1.0, levelBias: 0.0, timeScale: 1.0,
      doneAction: 0)
```

Arguments:

envelope	An Env instance, or an Array of Controls. (See Control and the example below for how to use this.) The envelope is polled when the EnvGen is triggered, and at the start of a new envelope segment. The Env inputs can be other UGens.
gate	This triggers the envelope and holds it open while > 0. If the Env is fixed-length (e.g. Env.linen, Env.perc), the gate argument is used as a simple trigger. If it is an sustaining envelope (e.g. Env.adsr, Env.asr), the envelope is held open until the gate becomes 0, at which point is released. If **gate** < 0, force release with time `-1.0 - gate`. See Forced release below.
levelScale	The levels of the breakpoints are multiplied by this value. This value can be modulated, but is only sampled at the start of a new envelope segment.
levelBias	This value is added as an offset to the levels of the breakpoints. This value can be modulated, but is only sampled at the start of a new envelope segment.
timeScale	The durations of the segments are multiplied by this value. This value can be modulated, but is only sampled at the start of a new envelope segment.
doneAction	An integer representing an action to be executed when the env is finished playing. This can be used to free the enclosing synth, etc. See Done for more detail.

Discussion:

NOTE: The actual minimum duration of a segment is not zero, but one sample step for audio rate and one block for control rate. This may result in asynchronicity when in two envelopes of different number of levels, the envelope times add up to the same total duration. Similarly, when modulating times, the new time is only updated at the end of the current segment - this may lead to asynchronicity of two envelopes with modulated times.

```
// as amplitude envelope
(
{
    var env = Env([0, 1, 0.5, 1, 0], [0.01, 0.5, 0.02, 0.5]);
    SinOsc.ar(470) * EnvGen.kr(env, doneAction: Done.freeSelf)
}.play
)
```

```
{
    var env = Env([0, 1, 0.5, 0.8, 0, 1.2, 0], [0.01, 0.5, 0.02, 0.5, 0.2,
0.5]);
    var gate = Impulse.kr(MouseX.kr(0.2, 3), 0.5);
    var gen = EnvGen.kr(env, gate);
    SinOsc.ar(270, SinOsc.ar(gen * 473)) * gen * 0.2
}.play
)
// EnvGen multichannel expands when passed a multichannel envelope
(
{
    SinOsc.ar(
        EnvGen.kr(
            Env.circle([0, 1, 0, (2..4), 0, LFNoise1.kr(0.1 ! 5) * 10, 0],
[0.01, 0.6])
        )
        * 240 + 300
    ).sum * 0.2
}.play;
)
```

Inherited class methods

7 methods from UGen ► show

8 methods from Object ► show

Instance Methods

Inherited instance methods

97 methods from UGen ► show

161 methods from AbstractFunction ► show

269 methods from Object ► show

Undocumented instance methods

.canFreeSynth

From extension in /usr/local/share/SuperCollider/SCClassLibrary/Common/Audio/canFreeSynth.sc

Examples

```
// retriggered envelope by Dust
(
{
    var env = Env([0.0, 0.5, 0.0, 1.0, 0.9, 0.0], [0.05, 0.1, 0.01, 1.0, 1.5],
-4);
    var envgen = EnvGen.ar(env, Dust.ar(1));
    SinOsc.ar(
        envgen * 1000 + 440
    ) * envgen * 0.1
}.play
);

// two channels
(
{
    var env = Env([0.0, [-0.2, 0.5], 0.0, 1.0, [-0.4, 0.9], 0.0], [0.05, 0.1,
0.01, 1.0, 1.5], -4);
```

```
        envgen * 440 + 550
    ) * envgen * 0.1
}.play
);

// an envelope in a SynthDef can be used to limit the synth's lifetime
(doneAction: Done.freeSelf)

(
SynthDef(\env_help, { |out, gate = 0, freq = 440|
    var z;
    z = EnvGen.kr(Env.perc, doneAction: Done.freeSelf) * SinOsc.ar(freq, 0, 0.1);
    Out.ar(out, z)
}).add;
)

(
fork {
    10.do {
        Synth(\env_help);
        0.2.rand.wait;
    }
}
)


// using a gated envelope to gate a sound:
(
SynthDef(\env_help, { |out, gate = 0, freq = 440, doneAction = 0|
    var z = EnvGen.kr(Env.adsr, gate, doneAction: doneAction) * SinOsc.ar(freq, 0,
0.1);
    Out.ar(out, z)
}).add;
)

a = Synth(\env_help);


// turn on
a.set(\gate, 1);

// turn off
a.set(\gate, 0);

// it does not matter to what value the gate is set, as long as it is > 0
a.set(\gate, 2);

a.set(\doneAction, 2, \gate, 0); // set doneAction to two to let the synth free
itself

a.free; // alternatively, free it directly.
```

Specifying an envelope for each new synth

```
(
SynthDef(\help_Env_newClear, { |out = 0|
    var env, envctl;
    // make an empty 4 segment envelope
    env = Env.newClear(4);
    // create a control argument array
    envctl = \env.kr(env.asArray);
    Out.ar(out,
        SinOsc.ar(EnvGen.kr(envctl, \gate.tr), 0, 0.3) // the gate control is a
trigger
    );
}).add;
```

```
Synth(\help_Env_newClear, [\gate, 1, \env, Env([700, 900, 900, 800], [1, 1, 1],
\exp)]); // 3 segments

// reset then play again:
Synth(\help_Env_newClear, [\gate, 1, \env, Env({ rrand(60, 70).midicps } ! 4, [1,
1, 1], \exp)]);

// the same written as an event:
(instrument: \help_Env_newClear, gate: 1, env: Env({ rrand(60, 70).midicps } ! 4,
[1, 1, 1], \exp)).play;
```

Forced release

If the gate of an EnvGen is set to -1 or below, then the envelope will cutoff immediately. The time for it to cutoff is the amount less than -1, with -1 being as fast as possible, -1.5 being a cutoff in 0.5 seconds, etc. The cutoff shape and final value are read from the Env's last node.

```
(
SynthDef(\stealMe, { |out, gate = 1|
    Out.ar(out, { BrownNoise.ar }.dup * EnvGen.kr(Env.asr, gate, doneAction:
Done.freeSelf))
}).add;
)

a = Synth(\stealMe);
a.release(3); //  // cutoff in 3 seconds

// this is how the OSC data looks like:
s.sendMsg(\s_new, \stealMe, 1001, 1, 0);
s.sendMsg(\n_set, 1001, \gate, -1.1); // cutoff in 0.1 seconds
```

If the synthDef has an arg named "gate", the convenience method of Node can be used: `node.release(releaseTime)`

```
d = { |gate = 1| { BrownNoise.ar }.dup * EnvGen.kr(Env.asr, gate, doneAction:
Done.freeSelf) }.play;
d.release(3);
```

Forced release ignores multi-node release stages, always performing a one-node release, reading curve and end value from the Env's last node, and overwriting its duration.

```
(
// a Synth with a multi-node release stage
d = { |gate = 1|
    var env = Env([0, 1, 0, 0.5, 0], 0.5, -4, releaseNode: 1);
    { BrownNoise.ar }.dup * EnvGen.kr(env, gate, doneAction: Done.freeSelf)
}.play;
)
// forced release in 2 seconds:
// end value (0) and shape (-4) are read from the env's last node, nodes 2 and 3
are skipped
d.release(2);

// without releaseTime: normal release stage, as defined in env (3 nodes)
d.release();
```

Fast triggering tests

```
(
{
    EnvGen.kr(
        Env.new([0.001, 1, 0.5, 0], [0.01, 0.3, 1], -4, 2, nil),
        Impulse.kr(10)
    ) * SinOsc.ar(440, 0, 0.1)
```

```
(
{
    EnvGen.kr(
        Env.perc(0.1, 0.0, 0.5, 1, \welch),
        Impulse.kr(100),
        timeScale: 0.1
    ) * SinOsc.ar(440, 0, 0.3)
}.play;
)
```

Modulating the levelScale

```
// no, it doesn't take a ugen in ...
(
{
    EnvGen.kr(
        Env.asr(0.1, 1.0, 0.5, \welch),
        1.0,
        FSinOsc.ar(1.0).range(0.0, 1.0),
        timeScale: 0.1
    ) * SinOsc.ar(440, 0, 0.3)
}.play;
)

// ...but an .ir rate input, a float or an ir rate ugen like Rand would work
(
{
    EnvGen.kr(
        Env.asr(0.1, 1.0, 0.5, \welch),
        1.0,
        Rand(0.1, 1.0),
        timeScale: 0.1
    ) * SinOsc.ar(440, 0, 0.3)
}.play;
)
```

Filter cutoff modulation and initial envelope level

When using an envelope to modulate a filter's cutoff/center frequency, at higher resonance settings, a short attack time can produce a spike in volume. If this effect is not desired, and the filter's attack is always at the beginning of the synth (as is usually the case when playing a pattern), the artifact can be avoided by setting the envelope's initial level closer to target level for shorter attack times (below about 0.05 sec).

The following example shows how to do this in a SynthDef: Attack times between 0 and 0.05 seconds will produce initial envelope values between 1.0 (attack time = 0 means no attack) and 0.0. If the attack time is greater than 0.05 seconds, the envelope will start at 0.0. (The envelope is between 0.0 and 1.0 and mapped onto a valid frequency range.)

To hear the (potentially **loud**) short-attack artifacts, edit the `var envInit` line to read `var envInit = 0;`.

```
(
a = Bus.control(s, 1);

Slider(nil, Rect(800, 200, 200, 25))
.action_({ |view| a.set(view.value.lincurve(0, 1, 0, 0.12, 4)) })
.front
.onClose_({ p.stop; a.free });

SynthDef(\test, { |out = 0, freq = 100, ffreq = 500, rq = 0.25, modFactor = 12,
atk = 0, dcy = 0.12|
    // change to 'envInit = 0' to hear the short-attack artifacts
    // warning: envInit = 0 may be LOUD
    var envInit = atk.linlin(0, 0.05, 1, 0);
    var filtEg = EnvGen.kr(Env([envInit, 1, 0], [atk, dcy], -4));
```

```
    ffreq = ffreq * (1 + (filtEg * modFactor));
    sig = BLowPass4.ar(sig, ffreq, rq) * ampEg;
    Out.ar(out, (sig * 0.1).dup)
}).add;

p = Pbind(
    \instrument, \test,
    \freq, 100,
    \atk, a.asMap
).play;
)
```

The above works only for initial values. If a short attack is retriggered in the middle of a synth, then the envelope cannot adjust the low value for reattack (because the envelope's initial value is used only once, at the beginning of the synth, and never touched again). In that case, open the amplitude envelope slightly later to avoid the artifact.

```
(
Slider(nil, Rect(800, 200, 200, 25))
.action_({ |view| z.set(\atk, view.value.lincurve(0, 1, 0, 0.12, 4)) })
.front
.onClose_({ z.free });

z = SynthDef(\test, { |out = 0, freq = 100, ffreq = 500, rq = 0.25, modFactor =
12, atk = 0, dcy = 0.12|
    var trig = Impulse.kr(1);
    var filtEg = EnvGen.kr(Env([0, 1, 0], [atk, dcy], -4), trig);
    var ampEnvDelay = max(0, (0.05 - atk) * 0.1);
    var ampEg = EnvGen.ar(Env.linen(0.001, 0.2, 0.1), TDelay.kr(trig,
ampEnvDelay));
    var sig = Saw.ar(freq);
    ffreq = ffreq * (1 + (filtEg * modFactor));
    sig = BLowPass4.ar(sig, ffreq, rq) * ampEg;
    Out.ar(out, (sig * 0.1).dup)
}).play;
)
```

NOTE: These artifacts are more prominent in SuperCollider versions after fall 2024 (3.14.x). In earlier versions, EnvGen's initial value was in the middle of the first segment, reducing the slope of the initial attack. Code migrated to newer SC versions may require one of the above adjustments.

More examples

For more information about the *control bus mapping* used in the line `a = Synth(\sine, [freq: f.asMap]);`, see Node: -map and Bus: -asMap.

```
// Changing an Env while playing
(
SynthDef(\env, { |i_outbus = 0|
    var env, envctl;

    // make a dummy 8 segment envelope
    env = Env.newClear(8);

    // create a control argument array
    envctl = \env.kr(env.asArray);

    ReplaceOut.kr(i_outbus, EnvGen.kr(envctl, doneAction: Done.freeSelf));
}).add;
)

(
SynthDef(\sine, { |out, freq = 440|
    Out.ar(out, SinOsc.ar(freq, 0, 0.2));
}).add;
)
```

```
// use f's control bus value for frequency
// i.e. *map* the control to read from the bus
a = Synth(\sine, [freq: f.asMap]);

Synth(\env, [i_outbus: f, env: Env([700, 900, 900, 800], [1, 1, 1]*0.4, \exp)]);

Synth(\env, [i_outbus: f, env: Env([1000, 1000, 800, 1000, 900, 1000], [1, 1, 1,
1, 1]*0.3, \step)]);

a.free;
f.free;
```

helpfile source: /usr/local/share/SuperCollider/HelpSource/Classes/EnvGen.schelp
link::Classes/EnvGen::